// Day 7 of 10

Day 7

10 Microservices Mistakes

Developers make these every day.
Don't be one of them.

→ Swipe through all 10

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #01

Starting with Microservices

Premature split = distributed monolith

```
// Bad
// Day 1: Split everything!
UserService / OrderService / PaymentService
// 3 devs, 10 services — wrong!

// Good
// Start monolith, extract when needed
// Rule: split only when you feel the pain
// Monolith first = Microservices right
```

💡 Monolith first, microservices when needed

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #02

No API Gateway

Clients calling every service directly

```
// Bad
Client → UserService:8081
Client → OrderService:8082
Client → PaymentService:8083

// Good
Client → API Gateway (8080)
         ├── /users  → UserService
         ├── /orders → OrderService
         └── /pay    → PaymentService
```

💡 API Gateway = single entry point always

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #03

Synchronous Everything

One slow service = entire chain slow

```
// Bad
OrderService → calls → PaymentService
             → calls → NotificationService
// Chain breaks if one is slow!

// Good
OrderService → Kafka topic
PaymentService    ← consumes event
NotificationService ← consumes event
// Async = resilient
```

💡 Use async messaging for non-critical flows

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #04

Shared Database

Defeats the whole purpose of microservices

```
// Bad
UserService ──┐
OrderService ─┼→ shared_db
PaymentService┘
// Tight coupling!

// Good
UserService    → users_db
OrderService   → orders_db
PaymentService → payments_db
// Each service owns its data
```

💡 One service = one database, always

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #05

No Circuit Breaker

One failing service crashes everything

```
// Bad
OrderService calls PaymentService
PaymentService is DOWN
OrderService keeps retrying → crashes
// Cascading failure

// Good
@CircuitBreaker(name="payment",
  fallbackMethod="paymentFallback")
public String processPayment( ... ){ ... }
// Resilience4j handles failures
```

💡 Circuit breaker = bulkhead for failures

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #06

No Centralized Logging

Debugging across services is nightmare

```
// Bad
// UserService logs → server1/logs
// OrderService logs → server2/logs
// Good luck debugging a request!

// Good
// ELK Stack or similar
// Add correlationId to every request
log.info("Order created correlationId={}",id);
// Trace one request across all services
```

💡 Correlation ID is mandatory

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #07

No Service Discovery

Hardcoded IPs = unmaintainable

```
// Bad
String url = "http://192.168.1.10:8082/pay";
// IP changes = everything breaks!

// Good
// Eureka / Consul for discovery
String url = "http://PAYMENT-SERVICE/pay";
// Service name, not hardcoded IP
```

💡 Use Eureka or Consul for service discovery

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #08

Missing Health Checks

Dead service still gets traffic

```
// Bad
// No health endpoint
// Load balancer sends traffic to dead pod
// Users get 500 errors

// Good
// Spring Boot Actuator
management.endpoints.web.exposure.include=health
// Returns UP/DOWN status
// LB checks before routing
```

💡 Every service needs /actuator/health

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #09

No Distributed Tracing

Can't trace a request across services

```
// Bad
// User reports: 'order failed'
// Which service failed? No idea!
// Check 5 log files manually

// Good
// Zipkin + Sleuth
// Every request gets traceId + spanId
// See full journey in Zipkin UI:
// Gateway → Order → Payment → Notify
```

💡 Zipkin + Sleuth = full request tracing

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #10

Ignoring Idempotency

Duplicate requests = duplicate orders

```
// Bad
// Payment called twice on retry
// User charged twice!
// No duplicate check

// Good
POST /payments
Idempotency-Key: order-123-pay-1
// Same key = same result
// No duplicate charges
```

💡 Every payment API must be idempotent

Shubham Bhati
linkedin.com/in/bhatishubham

```
done = true
```

Follow for more tips!

Like · Comment · Repost

Shubham Bhati
linkedin.com/in/bhatishubham